

SISTEMA DE VAULT COMPARTIDO

Especificacion funcional y tecnica para integracion en el game engine

Objetivo: implementar un almacen global compartido entre todos los personajes de una misma partida/perfil. El Vault debe aceptar cualquier categoria de objeto soportada por el juego y permitir transferencias seguras, bidireccionales y persistentes entre el inventario activo de un personaje y el almacenamiento compartido.

1. Principio general

El Vault no pertenece a ningun personaje. Pertenece al estado global de la partida o cuenta de progreso correspondiente. Todos los personajes autorizados leen y modifican la misma instancia persistente de Vault.

La regla central es: un objeto solo puede estar en una ubicacion logica a la vez. Si se transfiere del inventario al Vault, debe retirarse del inventario antes de confirmar la operacion; si la operacion falla, debe revertirse sin duplicar ni perder el objeto.

2. Alcance del Vault

Categoria	Comportamiento sugerido
Armas	Guardar instancias completas, incluyendo nombre, Tier, material, stats procedurales, afijos y durabilidad.
Armaduras	Guardar instancia y estado completo.
Consumibles	Permitir stacking por item_id cuando sus propiedades sean identicas.
Materiales de crafting	Stacking por tipo/material; cantidades potencialmente altas.
Componentes / afijos	Guardar como objetos independientes hasta usarse en crafting.
Objetos de mision	Soporte configurable: transferibles, bloqueados o excluidos del Vault.
Moneda / recursos	Preferible sistema separado si son valores numericos globales; opcionalmente pueden modelarse como items.

3. Responsabilidades del sistema

- Mantener una unica instancia de Vault compartida entre personajes.
- Almacenar objetos heterogeneos usando una estructura comun de ItemInstance.
- Permitir deposito: Inventario -> Vault.
- Permitir retiro: Vault -> Inventario.
- Validar capacidad, reglas de transferencia y stacking antes de modificar datos.
- Persistir el estado del Vault independientemente del personaje activo.
- Evitar duplicacion o perdida de objetos ante errores.
- Permitir filtros, ordenamiento y busqueda sin alterar los datos subyacentes.

4. Arquitectura de datos

Se recomienda separar tres conceptos: **ItemDefinition** para datos estaticos, **ItemInstance** para objetos concretos y **Container** para inventarios y Vault. El Vault y el inventario deben reutilizar la misma logica base de contenedor.

ItemDefinition

```
ItemDefinition = {  
  "item_id": "steel_sword",  
  "category": "weapon",  
  "stackable": False,  
  "max_stack": 1,  
  "vault_allowed": True,  
  "base_data": {...}  
}
```

ItemInstance

```
ItemInstance = {  
  "instance_id": "uuid",  
  "item_id": "steel_sword",  
  "quantity": 1,  
  "custom_data": {  
    "name": "La Ultima Razon",  
    "tier": 3,  
    "material": "steel",  
    "stats": {...},  
    "affix": {...},  
    "durability": 82  
  }  
}
```

Contenedores

```
CharacterInventory = {  
  "owner_character_id": "char_001",  
  "capacity": 30,  
  "items": [ItemInstance, ...]  
}  
  
SharedVault = {  
  "vault_id": "global_vault",  
  "capacity": 200,  
  "items": [ItemInstance, ...]  
}
```

5. Vault compartido entre personajes

El Vault debe cargarse desde el estado global al entrar a la zona/interfaz correspondiente. Cambiar de personaje no crea otro Vault. El nuevo personaje accede al mismo conjunto de ItemInstance.

```
SAVE_ROOT
|-- world_state
|-- characters
|   |-- char_001_inventory
|   |-- char_002_inventory
|   |-- char_003_inventory
|-- shared_vault
```

Esto permite que un personaje consiga un arma, material o componente en Dungeon, lo deposite y otro personaje lo retire posteriormente.

6. Mecanica de transferencia

La interfaz debe soportar transferencia bidireccional. El jugador selecciona un objeto en uno de los paneles y ejecuta Depositar o Retirar. Para objetos apilables debe poder mover toda la pila o una cantidad especifica.

Accion	Origen	Destino	Validaciones principales
Depositar	Inventario del personaje	Vault	vault_allowed, cantidad valida, espacio/stack disponible
Retirar	Vault	Inventario del personaje	cantidad valida, capacidad/peso/stack del inventario

Flujo atomico recomendado

1. Seleccionar ItemInstance y cantidad.
2. Validar que el origen contiene esa instancia/cantidad.
3. Validar que el destino puede aceptarla.
4. Preparar la operacion sin modificar aun el save persistente.
5. Remover o reducir la cantidad en el origen.
6. Insertar o sumar la cantidad en el destino.
7. Confirmar la transaccion.
8. Persistir ambos estados.
9. Si cualquier paso falla, ejecutar rollback.

La transferencia debe ser tratada como una sola transaccion. Nunca debe guardarse el origen modificado si el destino no pudo aceptar el objeto.

7. Stacking

Los objetos apilables pueden compartir un slot solo cuando son logicamente equivalentes. Para materiales y consumibles simples basta normalmente con item_id. Para objetos con propiedades variables se requiere una clave de stacking mas estricta.

```
def stack_key(item):
    if not item.definition.stackable:
        return item.instance_id
    return (
        item.item_id,
        normalizar_custom_data_relevante(item.custom_data)
    )
```

Ejemplo: dos pociones identicas pueden apilarse. Dos espadas de acero Tier III con stats procedurales diferentes no deben apilarse, aunque compartan item_id.

8. Capacidad

El Vault puede implementarse con slots limitados, capacidad por peso o capacidad virtualmente ilimitada. Para una primera version se recomienda **capacidad por slots**, porque es simple de comunicar y validar.

Opcion	Ventaja	Riesgo
Slots limitados	Simple, visible, facil de balancear	Requiere expansion o gestion
Peso	Mas simulacion y decisiones	Complejiza la UI y las transferencias
Ilimitado	Cero friccion	Reduce decisiones y puede acumular miles de instancias

Recomendacion inicial: Vault con capacidad amplia por slots y posibilidad de ampliacion futura. Los stacks ocupan un solo slot.

9. Reglas especiales de transferencia

- Objetos equipados no pueden depositarse sin desequiparse primero.
- Objetos marcados `quest_locked` o `bound` pueden bloquear la transferencia segun diseño.
- Un objeto usado actualmente por una accion/crafting no puede moverse hasta finalizar esa operacion.
- No aceptar `quantity <= 0` ni `quantity` superior a la disponible.
- Las armas y objetos unicos conservan exactamente su `instance_id` y `custom_data` al moverse.
- Un objeto retirado por otro personaje no se copia: cambia de contenedor.
- La UI nunca debe ser la fuente de verdad; el backend del sistema debe revalidar la operacion.

10. Pseudocodigo de transferencia

```
def transfer_item(source, destination, instance_id, quantity):

    item = source.find(instance_id)
    if item is None:
        return TransferResult.fail("ITEM_NOT_FOUND")

    if quantity <= 0 or quantity > item.quantity:
        return TransferResult.fail("INVALID_QUANTITY")

    if not destination.accepts(item, quantity):
        return TransferResult.fail("DESTINATION_REJECTED")

    snapshot_source = source.snapshot()
    snapshot_destination = destination.snapshot()

    try:
        moved_item = source.remove(instance_id, quantity)
        destination.add(moved_item)
        save_transaction(source, destination)
        return TransferResult.success(moved_item)

    except Exception:
        source.restore(snapshot_source)
        destination.restore(snapshot_destination)
        return TransferResult.fail("TRANSFER_ROLLBACK")
```

11. API sugerida

Funcion	Responsabilidad
vault.get_items()	Devuelve contenido actual del Vault.
vault.can_accept(item, qty)	Valida capacidad y reglas.
inventory.can_accept(item, qty)	Valida inventario del personaje.
transfer_to_vault(character_id, instance_id, qty)	Deposito.
transfer_from_vault(character_id, instance_id, qty)	Retiro.
split_stack(instance_id, qty)	Crea una subcantidad transferible sin duplicar contenido.
merge_stacks()	Consolida stacks compatibles.
sort/filter/search	Solo vista; no modifica identidad de las instancias.

12. Interfaz de usuario

La pantalla recomendada utiliza dos paneles simultaneos: **Inventario del personaje** a la izquierda y **Vault compartido** a la derecha. El jugador selecciona un objeto y lo mueve mediante botones, doble clic, drag-and-drop o una combinacion de estos metodos.

```
INVENTARIO DEL PERSONAJE          VAULT COMPARTIDO
[ Arma ] [ Pocion x5 ]             [ Material x24 ] [ Daga ]
[ Componente ] [ Armadura ] <->  [ Espada ] [ Pocion x12 ]

Acciones:
- Depositar
- Retirar
- Mover cantidad
- Depositar todo / Retirar todo (opcional)
- Filtrar por categoria
- Buscar por nombre
- Ordenar
```

13. Persistencia y guardado

El Vault debe persistirse como parte del estado global y no dentro del archivo de un personaje individual. Las transferencias deben guardar de forma consistente tanto el Vault como el inventario afectado.

Si el engine soporta guardado transaccional, usar una transaccion unica. Si no, mantener una copia temporal/snapshot y aplicar rollback ante fallo de serializacion.

14. Integracion con crafting

El Vault puede convertirse en el puente natural entre Dungeon y crafting. Los materiales y componentes recogidos por cualquier personaje pueden depositarse y quedar disponibles para otros personajes o para una estacion de crafting global.

Para evitar ambigüedad, definir explícitamente si el sistema de crafting consume recursos solo del inventario activo o si puede consumir directamente del Vault. Recomendacion para primera version: crafting consume del inventario del personaje; el jugador debe retirar previamente los materiales. Esto mantiene visible y comprensible el flujo de objetos.

15. Casos de prueba minimos

- Depositar un arma unica y retirarla con otro personaje conservando instance_id y todas sus propiedades.
- Depositar parte de un stack de 20 materiales y verificar que quedan 15 en inventario y 5 en Vault.
- Depositar un stack sobre otro compatible y comprobar merge sin crear slot extra.
- Intentar retirar un objeto con inventario lleno y confirmar que el Vault no cambia.
- Intentar depositar un objeto bloqueado y confirmar rechazo.
- Provocar un fallo durante la transferencia y verificar rollback completo.
- Guardar, cerrar, cargar con otro personaje y comprobar que el contenido del Vault persiste.
- Transferir un arma procedural con nombre personalizado, material y afijo y comprobar integridad total.

16. Resultado esperado

El sistema final debe funcionar como una capa global de almacenamiento, independiente de cada personaje, pero compatible con el mismo modelo de items e inventarios. El jugador puede conseguir objetos con un personaje, depositarlos en el Vault y recuperarlos con cualquier otro, sin conversiones, duplicados ni perdida de datos.

Arquitectura clave: ItemDefinition define el objeto, ItemInstance representa la copia concreta, Inventory y Vault son contenedores, y TransferService es el unico responsable de mover objetos de manera atomica entre ellos.